

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 9
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2026

Поставка завдання

Task 1. Parallel algorithm and shared state

Переказ коштів

У Вас є клас Bank з методом transfer () для переказу грошей з одного рахунку на інший в межах банку.

```
public class Bank {  
    public void transfer(Account from, Account to, int amount)  
    { atomic { from.withdraw(amount); to.deposit(amount); } } }
```

На жаль, в Java відсутня конструкція типу atomic для виконання транзакцій.

Напишіть свою реалізація тіла методу transfer(), яка могла б працювати в багатопотоковому середовищі.

Під час переказу грошей з рахунку на рахунок, дані рахунки повинні блокуватися.

Подумайте, як при цьому уникнути deadlock-ів.

Виконайте тестування методу transfer().

Для цього:

1 Створіть кілька десятків (сотень) рахунків і покладіть на них випадкову кількість грошей

2 Підрахуйте скільки всього є грошей (сума грошей на всіх рахунках)

3 Запустіть в декількох тисячах потоків одночасний переказ випадкових сум грошей з випадкового рахунку на випадковий рахунок (сума на рахунку не може бути негативною)

Дочекайтеся закінчення переказів і підрахуйте скільки грошей є всього в банку (сума грошей на всіх рахунках). Сума грошей в банку до переказів і після, очевидно, повинні збігатися.

Task 2. Producer-consumer task

Реалізувати потокобезпечний кільцевий буфер та виконання наступних вимог:

1) П'ять потоків генерують рядки в кільцевий буфер. Рядок має формат: «Потік № ... згенерував повідомлення ...»

2) Інші два потоки перекладають рядки з першого кільцевого у другий кільцевий буфер. У другий кільцевий буфер записується рядок наступного формату: «Потік № ... переклав повідомлення ...».

3) Основний потік програми вичитує та роздруковує 100 повідомлень із другого буфера.

Потоки, описані у вимогах 1)-2) повинні бути потоками-демонами, інакше програма не завершить своє виконання.

Кільцевий буфер – це зв'язний список, який замкнений у кільце. Кільцевий буфер зберігає значення за допомогою двох індексів: головного та кінцевого. При занесенні даних в буфер збільшується кінцевий індекс. При видобуванні даних із буфера збільшується головний індекс. При виході за границю списку індекси автоматично корегуються, переходячи до першого елементу списку. Ситуація, коли головний елемент співпадає із кінцевим, інтерпретується як порожній буфер.

При роботі з кільцевим буфером врахувати наступне:

1) якщо кільцевий буфер порожній, то потік, що видобуває дані, повинен зачекати, поки у буфері з'являться дані;

2) якщо кільцевий буфер повний, то потік, що додає дані, повинен зачекати, поки у буфері не з'явиться місце.

Виконання завдання 1

Для виконання завдання я використала механізм синхронізації об'єктів (Intrinsic Locks) та стратегію впорядкування ресурсів. Основна складність полягає в тому, що при одночасному переказі від А до В та від В до А може виникнути ситуація, коли два потоки вічно чекатимуть на звільнення рахунків один одного.

Ключові моменти реалізації:

- Запобігання Deadlock: Для уникнення взаємного блокування я впровадила фіксований порядок захоплення замків. Незалежно від того, хто є відправником, а хто отримувачем, програма спочатку блокує рахунок з меншим ID, а потім з більшим. Це гарантує, що всі потоки завжди намагатимуться отримати доступ до ресурсів у однаковій послідовності.

- Синхронізація: Використано вкладені блоки synchronized. Це дозволяє «заморозити» стан обох рахунків на час проведення транзакції, забезпечуючи її атомарність.

- Тестування: Використано пул потоків ExecutorService (FixedThreadPool) для імітації високого навантаження. Для генерації випадкових даних у багатопотоковому середовищі використано ThreadLocalRandom, що працює швидше та ефективніше за звичайний Random.

- Цілісність даних: Програма порівнює сумарний баланс усіх рахунків до початку роботи та після завершення всіх 10 000 операцій. Результат

Success: true підтверджує відсутність втрати або «магічної появи» грошей під час паралельних обчислень.

Скріншоти коду

```
class Account {
    private final int id;
    private int balance;

    public Account(int id, int balance) {
        this.id = id;
        this.balance = balance;
    }

    public int getId() {
        return id;
    }

    public int getBalance() {
        return balance;
    }

    void withdraw(int amount) {
        balance -= amount;
    }

    void deposit(int amount) {
        balance += amount;
    }
}
```

```
class Bank {
    public void transfer(Account from, Account to, int amount) {
        if (from == to) return;

        Account first = from.getId() < to.getId() ? from : to;
        Account second = from.getId() < to.getId() ? to : from;

        synchronized (first) {
            synchronized (second) {
                if (from.getBalance() >= amount) {
                    from.withdraw(amount);
                    to.deposit(amount);
                }
            }
        }
    }
}
```

```

public class Main {

    public static void main(String[] args) throws InterruptedException {

        int numAccounts = 100;
        int initialBalance = 1000;
        int numTransfers = 10_000;

        Bank bank = new Bank();
        List<Account> accounts = new ArrayList<>();

        // Створення рахунків
        for (int i = 0; i < numAccounts; i++) {
            accounts.add(new Account(i, initialBalance));
        }

        // Підрахунок суми до переказів
        long totalBefore = accounts.stream()
            .mapToLong(Account::getBalance)
            .sum();

        System.out.println("Total before: " + totalBefore);

        ExecutorService executor = Executors.newFixedThreadPool(10);

        for (int i = 0; i < numTransfers; i++) {
            executor.execute(() -> {
                ThreadLocalRandom rand = ThreadLocalRandom.current();

                Account from = accounts.get(rand.nextInt(numAccounts));
                Account to = accounts.get(rand.nextInt(numAccounts));
                int amount = rand.nextInt(50);

                bank.transfer(from, to, amount);
            });
        }

        executor.shutdown();
        executor.awaitTermination(1, TimeUnit.MINUTES);

        // Підрахунок суми після переказів
        long totalAfter = accounts.stream()
            .mapToLong(Account::getBalance)
            .sum();

        System.out.println("Total after: " + totalAfter);
        System.out.println("Success: " + (totalBefore == totalAfter));
    }
}

```

Скріншоти виконання коду

```

Total before: 100000
Total after: 100000
Success: true

```

Виконання завдання 2

Для вирішення я розробила клас `CircularBuffer` на основі масиву, де рух по колу реалізовано за допомогою операцій з індексами `head` та `tail`.

Ключові моменти реалізації:

- Механізм очікування (`Guarded Suspension`): Замість "холостого" циклу, який би навантажував процесор, я використала методи `wait()` та `notifyAll()`. Якщо буфер заповнений, потік-виробник засинає, поки споживач не звільнить місце.

- Циклічність: Перехід індексів до початку масиву після досягнення кінця реалізовано через оператор залишку від ділення: $(index + 1) \% buffer.length$. Це дозволяє ефективно використовувати пам'ять без постійного перестворення об'єктів.

- Потоки-демони (`Daemon Threads`): Оскільки виробники та транслятори працюють у нескінченних циклах `while(true)`, я встановила їм статус `setDaemon(true)`. Це дозволяє JVM автоматично завершити їх роботу, як тільки основний потік (`main`) закінчить вичитувати 100 повідомлень.

- Потокобезпека: Всі методи доступу до буфера є `synchronized`, що виключає стан гонитви (`race condition`) при одночасній зміні лічильника `count` різними потоками.

Скріншоти коду

```
class CircularBuffer {
    private final String[] buffer;
    private int head = 0;
    private int tail = 0;
    private int count = 0;

    public CircularBuffer(int size) {
        this.buffer = new String[size];
    }

    public synchronized void put(String item) throws InterruptedException {
        while (count == buffer.length) {
            wait();
        }
        buffer[tail] = item;
        tail = (tail + 1) % buffer.length;
        count++;
        notifyAll();
    }

    public synchronized String take() throws InterruptedException {
        while (count == 0) {
            wait();
        }
        String item = buffer[head];
        head = (head + 1) % buffer.length;
        count--;
        notifyAll();
        return item;
    }
}
```

```

public class ProducerConsumerTask {
    public static void main(String[] args) throws InterruptedException {
        CircularBuffer buffer1 = new CircularBuffer(10);
        CircularBuffer buffer2 = new CircularBuffer(10);

        for (int i = 1; i <= 5; i++) {
            int threadId = i;
            Thread t = new Thread(() -> {
                try {
                    while (true) {
                        buffer1.put("Потік № " + threadId + " згенерував повідомлення");
                        Thread.sleep(50);
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            t.setDaemon(true);
            t.start();
        }

        for (int i = 1; i <= 2; i++) {
            int threadId = i;
            Thread t = new Thread(() -> {
                try {
                    while (true) {
                        String msg = buffer1.take();
                        buffer2.put("Потік № " + threadId + " переклав повідомлення (" + msg + ")");
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
            t.setDaemon(true);
            t.start();
        }

        for (int i = 1; i <= 100; i++) {
            System.out.println(i + ": " + buffer2.take());
        }

        System.out.println("Програма завершена.");
    }
}

```

Скріншоти виконання коду

```

1: Потік № 2 переклав повідомлення (Потік № 5 згенерував повідомлення)
2: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
3: Потік № 2 переклав повідомлення (Потік № 4 згенерував повідомлення)
4: Потік № 1 переклав повідомлення (Потік № 1 згенерував повідомлення)
5: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
6: Потік № 1 переклав повідомлення (Потік № 5 згенерував повідомлення)
7: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
8: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
9: Потік № 2 переклав повідомлення (Потік № 4 згенерував повідомлення)
10: Потік № 1 переклав повідомлення (Потік № 1 згенерував повідомлення)
11: Потік № 2 переклав повідомлення (Потік № 5 згенерував повідомлення)
12: Потік № 1 переклав повідомлення (Потік № 3 згенерував повідомлення)
13: Потік № 1 переклав повідомлення (Потік № 1 згенерував повідомлення)
14: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
15: Потік № 1 переклав повідомлення (Потік № 4 згенерував повідомлення)
16: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
17: Потік № 1 переклав повідомлення (Потік № 5 згенерував повідомлення)
18: Потік № 1 переклав повідомлення (Потік № 1 згенерував повідомлення)

```

85: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
86: Потік № 1 переклав повідомлення (Потік № 5 згенерував повідомлення)
87: Потік № 2 переклав повідомлення (Потік № 4 згенерував повідомлення)
88: Потік № 2 переклав повідомлення (Потік № 1 згенерував повідомлення)
89: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
90: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
91: Потік № 1 переклав повідомлення (Потік № 5 згенерував повідомлення)
92: Потік № 2 переклав повідомлення (Потік № 4 згенерував повідомлення)
93: Потік № 2 переклав повідомлення (Потік № 1 згенерував повідомлення)
94: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
95: Потік № 2 переклав повідомлення (Потік № 2 згенерував повідомлення)
96: Потік № 2 переклав повідомлення (Потік № 4 згенерував повідомлення)
97: Потік № 1 переклав повідомлення (Потік № 5 згенерував повідомлення)
98: Потік № 2 переклав повідомлення (Потік № 1 згенерував повідомлення)
99: Потік № 2 переклав повідомлення (Потік № 3 згенерував повідомлення)
100: Потік № 1 переклав повідомлення (Потік № 2 згенерував повідомлення)